

Tutorial emmeans

Tutorial simple de emmeans

Ejemplo de trabajo

Primero armamos datos inventados.

```
# Datos inventados
datos <- data.frame(
  mosquitos = c(
    2, 3, 4,      # seca + natural
    8, 9, 10,     # lluviosa + natural
    5, 6, 7,      # post_lluvia + natural

    4, 5, 6,      # seca + artificial
    12, 14, 15,   # lluviosa + artificial
    8, 9, 10      # post_lluvia + artificial
  ),
  Season = c(
    "seca", "seca", "seca",
    "lluviosa", "lluviosa", "lluviosa",
    "post_lluvia", "post_lluvia", "post_lluvia",

    "seca", "seca", "seca",
    "lluviosa", "lluviosa", "lluviosa",
    "post_lluvia", "post_lluvia", "post_lluvia"
  ),
  Habitat = c(
    "natural", "natural", "natural",
    "natural", "natural", "natural",
    "natural", "natural", "natural",

    "artificial", "artificial", "artificial",
    "artificial", "artificial", "artificial",
    "artificial", "artificial", "artificial"
  )
)

datos
```

	mosquitos	Season	Habitat
1	2	seca	natural
2	3	seca	natural
3	4	seca	natural
4	8	lluviosa	natural
5	9	lluviosa	natural
6	10	lluviosa	natural
7	5	post_lluvia	natural
8	6	post_lluvia	natural

9	7	post_lluvia	natural
10	4	seca	artificial
11	5	seca	artificial
12	6	seca	artificial
13	12	lluviosa	artificial
14	14	lluviosa	artificial
15	15	lluviosa	artificial
16	8	post_lluvia	artificial
17	9	post_lluvia	artificial
18	10	post_lluvia	artificial

Tenemos tres estaciones:

seca
lluviosa
post_lluvia

Y dos tipos de hábitat:

natural
artificial

La respuesta es:

cantidad de mosquitos

1. Ajustar el modelo

Antes de ajustar el modelo, ordenamos los niveles.

```
datos$Season <- factor(  
  datos$Season,  
  levels = c("seca", "lluviosa", "post_lluvia")  
)  
  
datos$Habitat <- factor(  
  datos$Habitat,  
  levels = c("natural", "artificial")  
)  
  
levels(datos$Season)
```

```
[1] "seca"          "lluviosa"      "post_lluvia"
```

```
levels(datos$Habitat)
```

```
[1] "natural"      "artificial"
```

Esto hace que la referencia sea:

```
Season = seca
Habitat = natural
```

Ahora ajustamos el modelo:

```
modelo <- glm(
  mosquitos ~ Season + Habitat,
  family = poisson(link = "log"),
  data = datos
)

summary(modelo)
```

Call:

```
glm(formula = mosquitos ~ Season + Habitat, family = poisson(link = "log"),
    data = datos)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	1.1484	0.2300	4.994	5.92e-07	***
Seasonlluviosa	1.0415	0.2374	4.386	1.15e-05	***
Seasonpost_lluvia	0.6286	0.2528	2.487	0.0129	*
Habitatartificial	0.4299	0.1748	2.459	0.0139	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 30.3843 on 17 degrees of freedom
Residual deviance: 2.2547 on 14 degrees of freedom
AIC: 78.156

Number of Fisher Scoring iterations: 4

2. Interpretación del `summary()`

El `summary()` nos da algo como esto:

Coefficients:

	Estimate
(Intercept)	1.1484
Seasonlluviosa	1.0415
Seasonpost_lluvia	0.6286
Habitatartificial	0.4299

Al principio esto parece raro.

Uno espera ver:

seca + natural = tantos mosquitos
lluviosa + natural = tantos mosquitos
seca + artificial = tantos mosquitos

Pero el `summary()` no muestra eso directamente.

El `summary()` muestra piezas del modelo.

2.1 La escala log

El modelo usa:

```
family = poisson(link = "log")
```

Eso significa que el modelo no trabaja directamente con mosquitos.

Trabaja con:

$\log(\text{media esperada de mosquitos})$

Por eso los coeficientes no se leen como mosquitos.

Para volver a la escala de mosquitos usamos:

```
exp()
```

Regla simple:

$\exp(\text{coeficiente}) = \text{multiplicador}$

2.2 El intercepto

El intercepto es:

$(\text{Intercept}) = 1.1484$

Como estamos en escala log, hacemos:

```
exp(1.1484)
```

Eso da aproximadamente:

3.15

Interpretación:

En estación seca y hábitat natural se esperan unos 3.15 mosquitos.

¿Por qué seca y natural?

Porque esas son las categorías de referencia.

Season = seca
Habitat = natural

Entonces:

intercepto = seca + natural

2.3 Los coeficientes

Ahora miramos los otros coeficientes.

Estación lluviosa

Seasonlluviosa = 1.0415

Hacemos:

```
exp(1.0415)
```

Da aproximadamente:

2.83

Interpretación:

La estación lluviosa multiplica por 2.83 la cantidad esperada de mosquitos.

Entonces:

```
3.15 * 2.83
```

Da aproximadamente:

8.9

Interpretación:

En estación lluviosa y hábitat natural se esperan unos 8.9 mosquitos.

Estación post-lluvia

Seasonpost_lluvia = 0.6286

Hacemos:

```
exp(0.6286)
```

Da aproximadamente:

1.88

Interpretación:

La estación post-lluvia multiplica por 1.88 la cantidad esperada de mosquitos.

Entonces:

$3.15 * 1.88$

Da aproximadamente:

5.9

Interpretación:

En estación post-lluvia y hábitat natural se esperan unos 5.9 mosquitos.

Hábitat artificial

Habitatartificial = 0.4299

Hacemos:

$\exp(0.4299)$

Da aproximadamente:

1.54

Interpretación:

El hábitat artificial multiplica por 1.54 la cantidad esperada de mosquitos.

Entonces:

$3.15 * 1.54$

Da aproximadamente:

4.9

Interpretación:

En estación seca y hábitat artificial se esperan unos 4.9 mosquitos.

3. Completar la tabla a mano

Hasta acá tenemos estas piezas:

Referencia:

seca + natural = 3.15

Multiplicadores:

lluviosa = 2.83

post_lluvia = 1.88

artificial = 1.54

Ahora podemos completar la tabla.

Season	Habitat	Cuenta	Mosquitos esperados
seca	natural	3.15	3.15
lluviosa	natural	3.15×2.83	8.9
post_lluvia	natural	3.15×1.88	5.9
seca	artificial	3.15×1.54	4.9
lluviosa	artificial	$3.15 \times 2.83 \times 1.54$	13.7
post_lluvia	artificial	$3.15 \times 1.88 \times 1.54$	9.1

Esta tabla ya es mucho más fácil de entender que el `summary()`.

4. Limitación del `summary()`

El `summary()` no está mal.

Pero tiene una limitación:

no responde directamente cuántos mosquitos se esperan en cada grupo

El `summary()` da piezas:

intercepto

coeficientes

errores estándar

p-valores

Pero si alguien pregunta:

¿Cuántos mosquitos se esperan en lluviosa + artificial?

hay que hacer la cuenta:

$3.15 * 2.83 * 1.54$

Con dos variables todavía se puede hacer a mano.

Pero con más variables, más categorías y modelos más complejos, se vuelve incómodo.

Ahí aparece `emmeans`.

5. Uso de `emmeans`

5.1 Construir la tabla de medias esperadas

Le pedimos a `emmeans` la tabla completa:

```
library(emmeans)
```

Welcome to emmeans.

Caution: You lose important information if you filter this package's results.

See '? untidy'

```
emm_tabla <- emmeans(  
  modelo,  
  ~ Season * Habitat,  
  type = "response"  
)
```

```
emm_tabla
```

Season	Habitat	rate	SE	df	asympt.LCL	asympt.UCL
seca	natural	3.15	0.725	Inf	2.01	4.95
lluviosa	natural	8.93	1.440	Inf	6.52	12.25
post_lluvia	natural	5.91	1.080	Inf	4.13	8.46
seca	artificial	4.85	1.040	Inf	3.18	7.39
lluviosa	artificial	13.73	1.920	Inf	10.45	18.05
post_lluvia	artificial	9.09	1.490	Inf	6.59	12.54

Confidence level used: 0.95

Intervals are back-transformed from the log scale

Esto significa:

usá el modelo y armame la tabla de mosquitos esperados para cada combinación de Season y Habitat

La parte:

~ Season * Habitat

no cambia el modelo.

Solo le dice a `emmeans`:

mostrame todas las combinaciones de Season y Habitat

La parte:

type = "response"

significa:

devolveme los resultados en cantidad de mosquitos

Entonces:

summary() = piezas del modelo

emmeans() = tabla armada

5.2 Promedios solo por estación

También podemos pedir solo las estaciones:

```
emm_season <- emmeans(  
  modelo,  
  ~ Season,  
  type = "response"  
)  
  
emm_season
```

Season	rate	SE	df	asympt.LCL	asympt.UCL
seca	3.91	0.801	Inf	2.62	5.84
lluviosa	11.08	1.360	Inf	8.71	14.09
post_lluvia	7.33	1.100	Inf	5.46	9.84

Results are averaged over the levels of: Habitat

Confidence level used: 0.95

Intervals are back-transformed from the log scale

Esto devuelve una fila por estación:

```
seca  
lluviosa  
post_lluvia
```

Como el modelo también tiene **Habitat**, **emmeans** promedia los hábitats.

Entonces esta tabla responde:

¿Cuántos mosquitos se esperan por estación, promediando los hábitats?

5.3 Contrastes con **pairs()**

Después de tener las medias esperadas, puede aparecer otra pregunta:

¿Las estaciones son distintas entre sí?

Para eso usamos:

```
pairs(emm_season)
```

contrast	ratio	SE	df	null	z.ratio	p.value
seca / lluviosa	0.353	0.0838	Inf	1	-4.386	<0.0001
seca / post_lluvia	0.533	0.1350	Inf	1	-2.487	0.0344
lluviosa / post_lluvia	1.511	0.2900	Inf	1	2.148	0.0804

Results are averaged over the levels of: Habitat

P value adjustment: tukey method for comparing a family of 3 estimates

Tests are performed on the log scale

`pairs()` compara las filas de la tabla.

Si `emm_season` tiene:

```
seca  
lluviosa  
post_lluvia
```

entonces `pairs()` compara:

```
seca vs lluviosa  
seca vs post_lluvia  
lluviosa vs post_lluvia
```

Eso se llama contraste.

5.4 Qué es un contraste

Un contraste es una comparación entre dos medias.

En este caso:

```
seca vs lluviosa
```

pregunta:

```
¿seca y lluviosa tienen medias parecidas o diferentes?
```

Como este modelo usa link log, `pairs()` suele mostrar ratios.

Por ejemplo:

```
seca / lluviosa = 0.353
```

Eso significa:

```
seca tiene 0.353 veces los mosquitos de lluviosa
```

O al revés:

lluviosa tiene 2.83 veces los mosquitos de seca

Si el ratio fuera 1:

no habría diferencia

Regla simple:

ratio = 1 → iguales

ratio > 1 → el primero es mayor

ratio < 1 → el primero es menor

5.5 El p-valor del contraste

El p-valor del contraste ayuda a leer si la diferencia es clara o no.

Ejemplo:

seca / lluviosa = 0.353

p < 0.0001

Lectura:

seca y lluviosa son claramente diferentes

Otro ejemplo:

lluviosa / post_lluvia = 1.51

p = 0.0804

Lectura:

lluviosa parece mayor que post_lluvia,
pero la diferencia no es tan clara

5.6 Intervalos de confianza

`emmeans` también da intervalos.

Ejemplo:

Season	rate	asympt.LCL	asympt.UCL
seca	3.91	2.62	5.84
lluviosa	11.08	8.71	14.09
post_lluvia	7.33	5.46	9.84

Lectura:

rate = media estimada

asympt.LCL = límite inferior

asympt.UCL = límite superior

Ejemplo:

seca tiene una media estimada de 3.91 mosquitos,
con un intervalo entre 2.62 y 5.84

Esto recuerda que no estamos viendo números exactos.

Estamos viendo estimaciones.

5.7 Forest plot de medias esperadas

Hasta ahora vimos que `emmeans` puede armar una tabla.

Por ejemplo:

```
emm_season <- emmeans(  
  modelo,  
  ~ Season,  
  type = "response"  
)
```

emm_season

Season	rate	SE	df	asympt.LCL	asympt.UCL
seca	3.91	0.801	Inf	2.62	5.84
lluviosa	11.08	1.360	Inf	8.71	14.09
post_lluvia	7.33	1.100	Inf	5.46	9.84

Results are averaged over the levels of: Habitat
Confidence level used: 0.95
Intervals are back-transformed from the log scale

La columna importante es: rate = mosquitos esperados

Y los intervalos son:

asympt.LCL = límite inferior

asympt.UCL = límite superior

Entonces podemos hacer un gráfico.

La idea del gráfico es simple:

punto = media esperada

línea = intervalo de confianza

Primero pasamos el resultado a una tabla:

```
emm_season_df <- as.data.frame(emm_season)  
emm_season_df
```

Season	rate	SE	df	asympt.LCL	asympt.UCL
seca	3.909357	0.8012663	Inf	2.616026	5.842095
lluviosa	11.076512	1.3587720	Inf	8.709332	14.087086
post_lluvia	7.330045	1.1010847	Inf	5.460635	9.839434

Results are averaged over the levels of: Habitat
 Confidence level used: 0.95
 Intervals are back-transformed from the log scale

Ahora graficamos:

```
library(ggplot2)

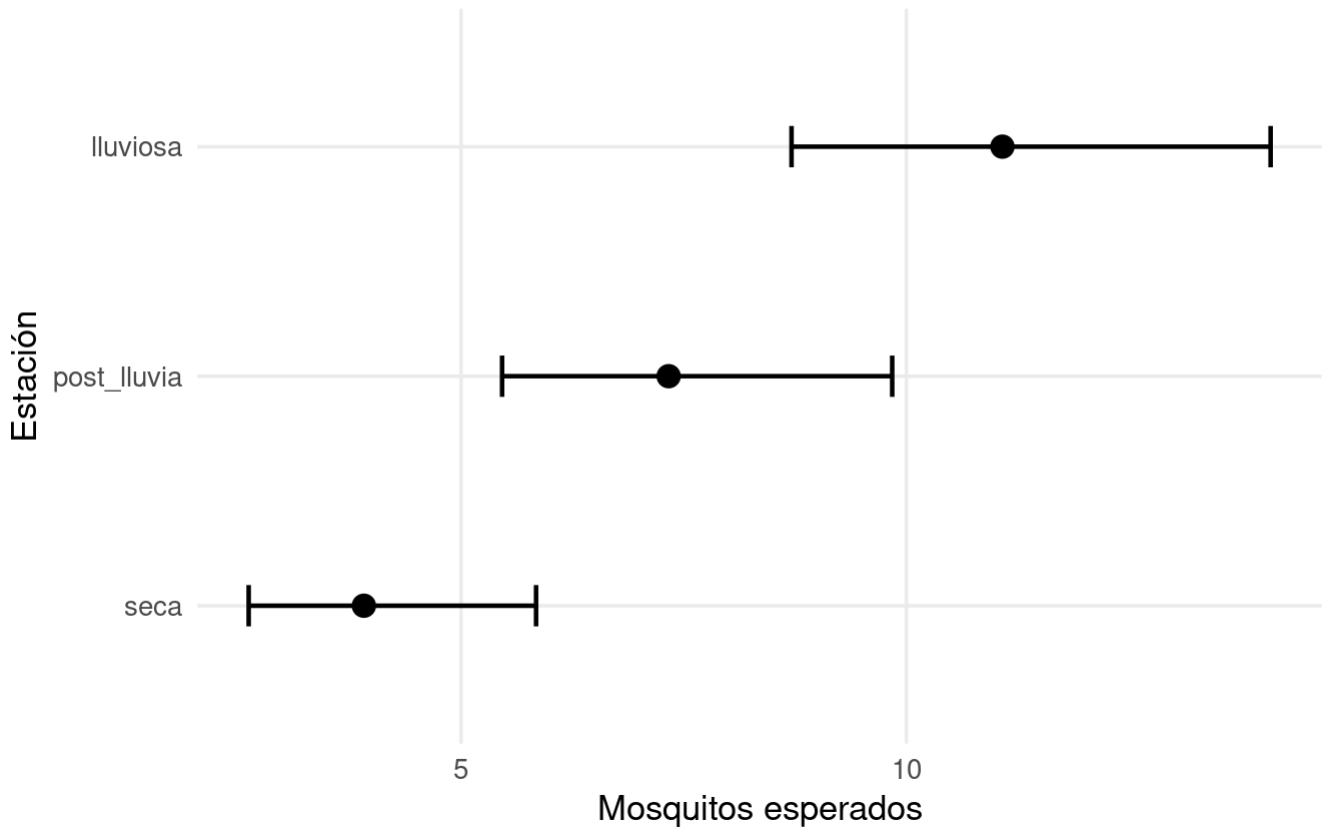
ggplot(
  emm_season_df,
  aes(
    x = rate,
    y = reorder(Season, rate)
  )
) +
  geom_errorbarh(
    aes(
      xmin = asympt.LCL,
      xmax = asympt.UCL
    ),
    height = 0.18,
    linewidth = 0.8
  ) +
  geom_point(size = 3.5) +
  labs(
    title = "Mosquitos esperados por estación",
    subtitle = "Medias estimadas por el modelo con intervalos de confianza del 95%",
    x = "Mosquitos esperados",
    y = "Estación"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
```

Warning: `geom\_errorbarh()` was deprecated in ggplot2 4.0.0.
 i Please use the `orientation` argument of `geom\_errorbar()` instead.

`height` was translated to `width`.

Mosquitos esperados por estación

Medias estimadas por el modelo con intervalos de confianza del 95%



Lectura del gráfico:

Cada punto es la media esperada de mosquitos para una estación. Cada línea muestra el intervalo de confianza.

Ejemplo:

Si lluviosa está más a la derecha, tiene más mosquitos esperados. Si el intervalo es más ancho, hay más incertidumbre.

5.8 Forest plot de contrastes

También podemos graficar los contrastes.

Primero hacemos los contrastes:

```
contr_season <- pairs(emm_season)
```

```
contr_season
```

contrast	ratio	SE	df	null	z.ratio	p.value
seca / lluviosa	0.353	0.0838	Inf	1	-4.386	<0.0001
seca / post_lluvia	0.533	0.1350	Inf	1	-2.487	0.0344
lluviosa / post_lluvia	1.511	0.2900	Inf	1	2.148	0.0804

Results are averaged over the levels of: Habitat
P value adjustment: tukey method for comparing a family of 3 estimates
Tests are performed on the log scale

Esto compara:

seca vs lluviosa

seca vs post_lluvia

lluviosa vs post_lluvia

Pasamos la salida a una tabla:

```
contr_season_df <- as.data.frame(  
  summary(contr_season, infer = c(TRUE, TRUE))  
)  
  
contr_season_df
```

contrast	ratio	SE	df	asympt.LCL	asympt.UCL	null
seca / lluviosa	0.3529412	0.08379848	Inf	0.2023176	0.6157026	1
seca / post_lluvia	0.5333333	0.13480667	Inf	0.2949326	0.9644387	1
lluviosa / post_lluvia	1.5111111	0.29038549	Inf	0.9631594	2.3707985	1

z.ratio	p.value
-4.386	<0.0001
-2.487	0.0344
2.148	0.0804

Results are averaged over the levels of: Habitat
Confidence level used: 0.95
Conf-level adjustment: tukey method for comparing a family of 3 estimates
Intervals are back-transformed from the log scale
P value adjustment: tukey method for comparing a family of 3 estimates
Tests are performed on the log scale

En esta salida, lo importante es:

contrast = comparación
ratio = cociente entre medias
asympt.LCL = límite inferior
asympt.UCL = límite superior

Ahora graficamos:

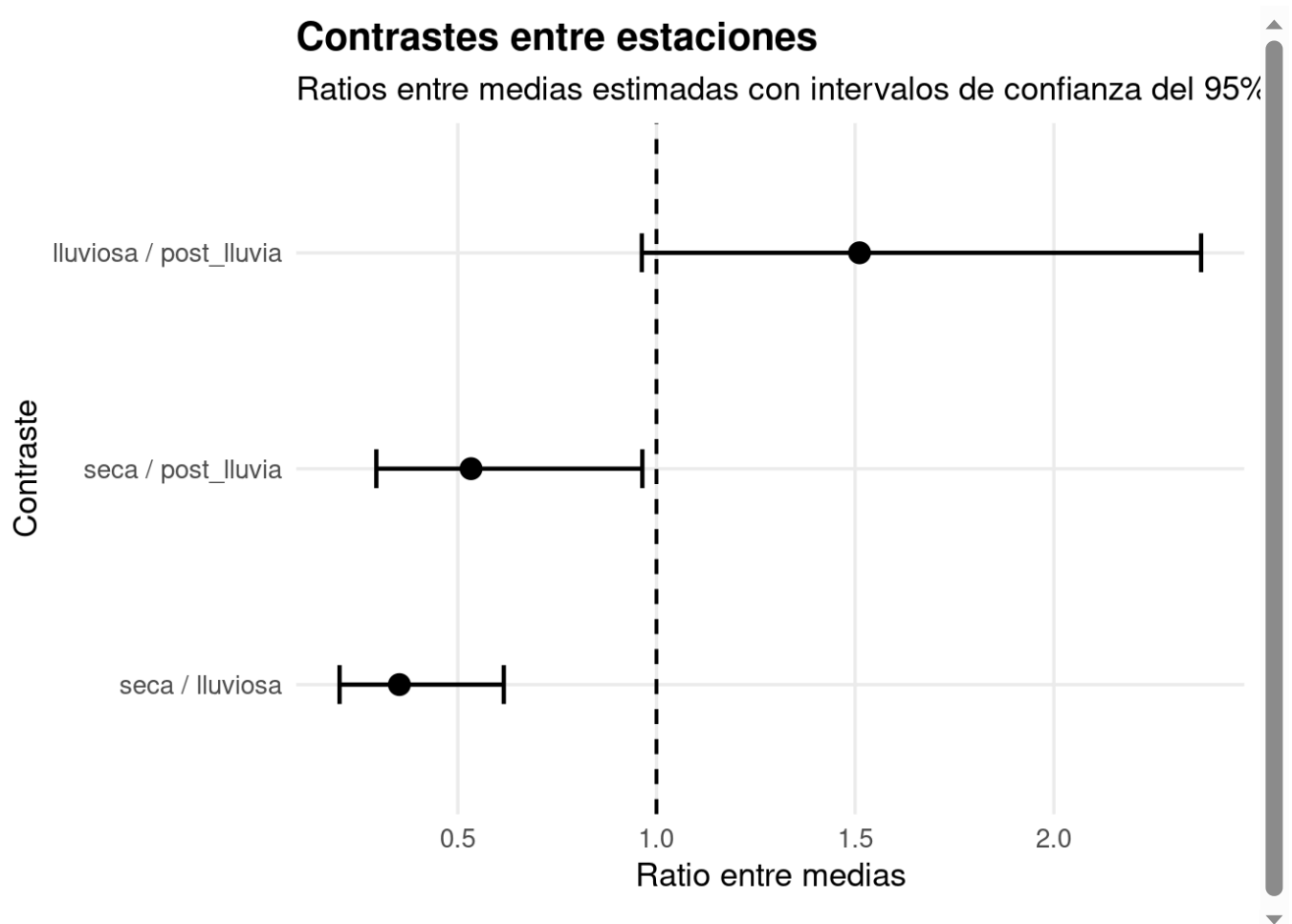
```
ggplot(  
  contr_season_df,  
  aes(  
    x = ratio,  
    y = reorder(contrast, ratio)  
  )  
)
```

```

) +
  geom_vline(
    xintercept = 1,
    linetype = "dashed",
    linewidth = 0.7
  ) +
  geom_errorbarh(
    aes(
      xmin = asymp.LCL,
      xmax = asymp.UCL
    ),
    height = 0.18,
    linewidth = 0.8
  ) +
  geom_point(size = 3.5) +
  labs(
    title = "Contrastes entre estaciones",
    subtitle = "Ratios entre medias estimadas con intervalos de confianza del 95%",
    x = "Ratio entre medias",
    y = "Contraste"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )

```

`height` was translated to `width`.



La línea vertical en 1 es importante.

ratio = 1 significa sin diferencia

Entonces:

si el intervalo cruza 1,
la diferencia no es clara

si el intervalo no cruza 1,
la diferencia es más clara

Ejemplo:

seca / lluviosa = 0.353

Significa:

seca tiene menos mosquitos que lluviosa

Si queremos leerlo al revés:

$1 / 0.353 = 2.83$

Entonces:

lluviosa tiene 2.83 veces los mosquitos de seca

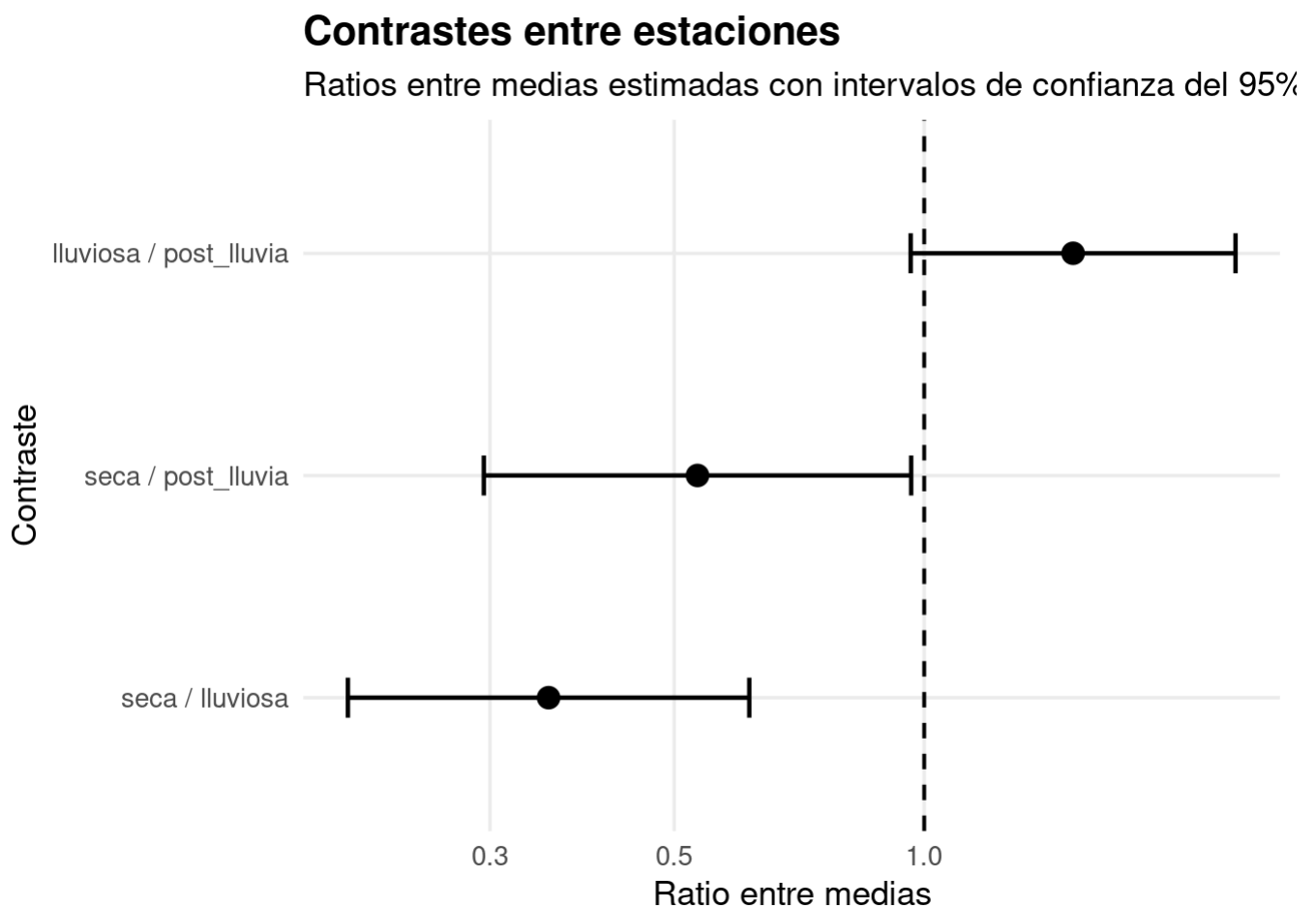
Mismo grafico en escala logaritmica

Como los ratios pueden ser menores o mayores que 1, muchas veces se ve mejor con escala logarítmica:

```
ggplot(
  contr_season_df,
  aes(
    x = ratio,
    y = reorder(contrast, ratio)
  )
) +
  geom_vline(
    xintercept = 1,
    linetype = "dashed",
    linewidth = 0.7
  ) +
  geom_errorbarh(
    aes(
      xmin = asymp.LCL,
      xmax = asymp.UCL
    ),
    height = 0.18,
    linewidth = 0.8
  ) +
  geom_point(size = 3.5) +
  scale_x_log10() +
```

```
labs(
  title = "Contrastes entre estaciones",
  subtitle = "Ratios entre medias estimadas con intervalos de confianza del 95%",
  x = "Ratio entre medias",
  y = "Contraste"
) +
theme_minimal(base_size = 13) +
theme(
  plot.title = element_text(face = "bold"),
  panel.grid.minor = element_blank()
)
```

`height` was translated to `width`.



6. Resumen final

La idea completa es esta:

```
summary()
= piezas del modelo
```

`exp(coeficiente)`
= multiplicador

`tabla a mano`
= combinamos referencia y multiplicadores

`emmeans()`
= arma esa tabla automáticamente

`pairs()`
= compara las medias de esa tabla

`contraste`
= comparación entre dos medias

En nuestro ejemplo:

`summary()` dice coeficientes en escala log

`emmeans()` dice cuántos mosquitos se esperan

`pairs()` dice qué grupos se diferencian